STAT 5845 Homework 7

November 10, 2025

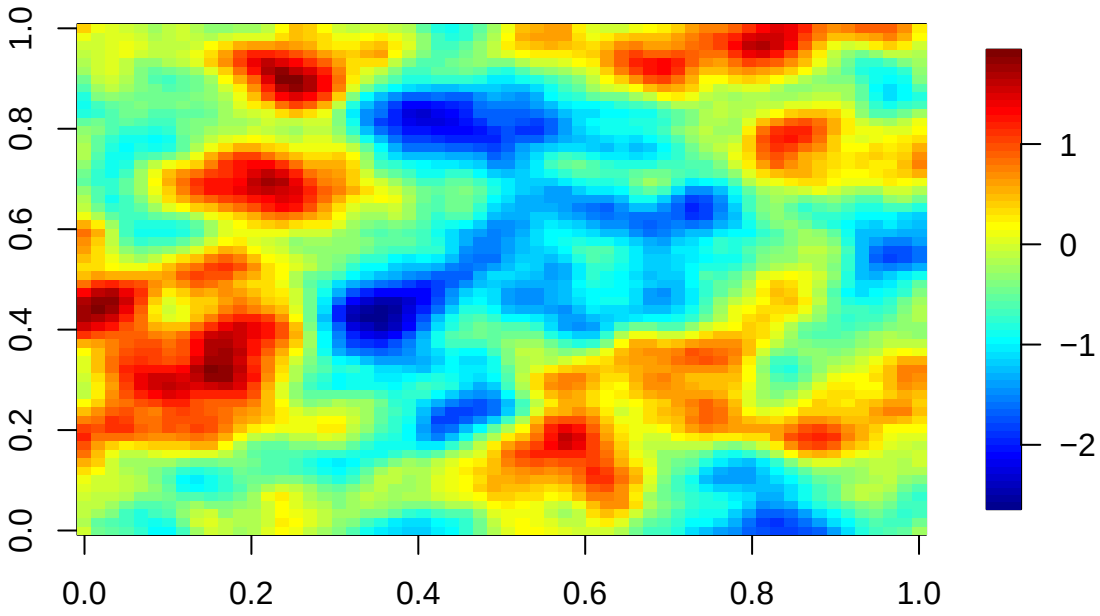
Contents

1 Preliminaries

1.1 Simulate a Gaussian random field on a 60 × 60 grid from a Matern covariance

```
# Start by building a 60x60 grid of locations
x <- seq(0, 1, length.out = 60)
y <- seq(0, 1, length.out = 60)
locs <- expand.grid(x = x, y = y) %>% as.matrix()
n.loc <- nrow(locs)

# True parameters to use for simulating
sigma2_true <- 1     # partial sill
beta_true   <- 0.05   # range parameter
nu_true     <- 2   # smoothness
tausq_true  <- 0     # nugget = 0 for simulation
mu_true     <- rep(0, n.loc)
```

```r
# Build distance matrix
distance_matrix <- as.matrix(dist(locs))

# Build Matern Covaraince
covariance_matrix <- (sigma2_true / (2^(nu_true - 1) * gamma(nu_true))) *
  (distance_matrix / beta_true)^nu_true * (
    besselK(distance_matrix / beta_true, nu_true)
  )
## Handle zero distances
diag(covariance_matrix) <- sigma2_true + tausq_true

# Set random seed
set.seed(124)

# Simulate Random Field using Matern Covariance Function
Z <- mvrnorm(1, mu = mu_true, Sigma = covariance_matrix)

quilt.plot(locs[,1], locs[,2], Z, nx = 60, ny = 60)
```



1.2 Randomly select 100 locations for training; use the remaining locations as testing.

```r
# Select 100 Random Points as training points
n.train <- 100
train_idx <- sample.int(n.loc, n.train) # randomly select 100 integers from 1
                                        # to n = number of grid points
```

```r
# Set the rest of the points in the grid to be used for kriging later
test_idx <- setdiff(seq_len(n.loc), train_idx)
train_locs <- locs[train_idx, , drop = FALSE]
train_Z <- Z[train_idx]
train_D <- as.matrix(dist(train_locs))
D_all_train <- fields::rdist(locs, train_locs)
```

2 Exponential

```r
# MLE Estimation, Exponential
#------------------- EXPONENTIAL -----------------------------------

# Note that exponential is a special case of Matern with nu = 0.5, so we can
# use the general matern MLE estimation structure, with some modifications

# Define negative log-likelihood function for MLE estimation via "optim"
nll_exponential <- function(par) {
  tausq   <- par[1]^2          # nugget variance (forced gte 0)
  sigmasq <- par[2]^2          # partial sill
  beta    <- exp(par[3])       # range parameter (gt 0)
  mu_val  <- par[4]            # constant mean

  # monitor parameter progress
  cat(sprintf("tau^2 = %.4f, sigma^2 = %.4f,
              beta = %.4f, mu = %.4f\n",
              tausq, sigmasq, beta, mu_val))

  mu <- rep(mu_val, n.loc)

  dist_mat <- train_D
  # Build covariance matrix
  C <- sigmasq * exp(-dist_mat / beta)
  diag(C) <- sigmasq + tausq

  # Cholesky decomposition for log-likelihood
  L <- t(chol(C))
  log.det.cov <- 2 * sum(log(diag(L)))
  z.new <- forwardsolve(L, train_Z - mu)
  inner.prod <- sum(z.new^2)

  # Negative Gaussian log-likelihood
  nll_value <- 0.5 * n.loc * log(2*pi) + 0.5 * log.det.cov + 0.5 * inner.prod
  return(nll_value)
```

```
}
#-------------------------------------------------------------#
# Fit parameters by minimizing NLL
init <- c(0.01, 1, log(0.1), 0)
```

```
fit <- optim(init, nll_exponential, control = list(trace = 0, maxit = 5000))
```

```
# Verify Convergence
str(fit)
```

```
## List of 5
##  $ par        : num [1:4] -0.000028 0.790774 -1.877661 -0.198057
##  $ value      : num 3300
##  $ counts     : Named int [1:2] 185 NA
##   ..- attr(*, "names")= chr [1:2] "function" "gradient"
##  $ convergence: int 0
##  $ message    : NULL
```

```
# Extract and print true vs estimated
est_par <- fit$par

results <- data.frame(
  Parameter = c("tau^2", "sigma^2", "beta (range)", "mu (mean)"),
  True = c(tausq_true, sigma2_true, beta_true, 0),
  Estimated = c(
    est_par[1]^2,
    est_par[2]^2,
    exp(est_par[3]),
    est_par[4]
  )
)

print("True vs. Estimated Parameters:")
```

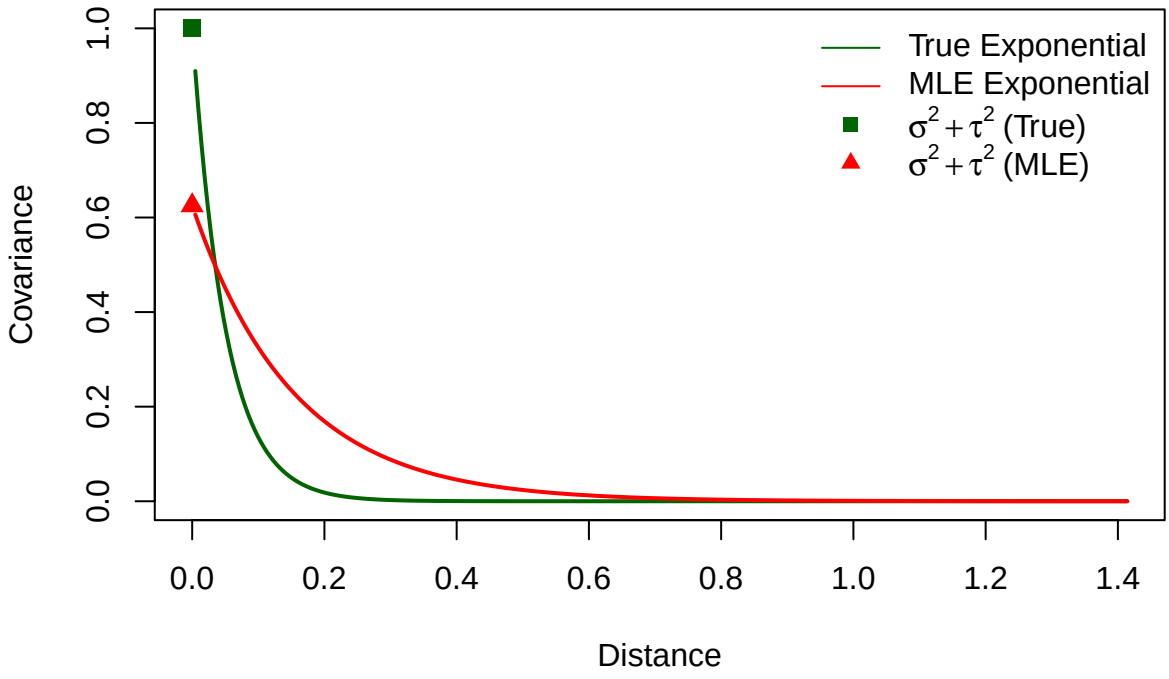
```
## [1] "True vs. Estimated Parameters:"
```

```
print(results, digits = 4, row.names = FALSE)
```

```
##     Parameter True  Estimated
##         tau^2 0.00  7.854e-10
##       sigma^2 1.00  6.253e-01
##  beta (range) 0.05  1.529e-01
##     mu (mean) 0.00 -1.981e-01
```

```
# Plot Exponential covariance - True vs MLE

# Distance grid for curves
```

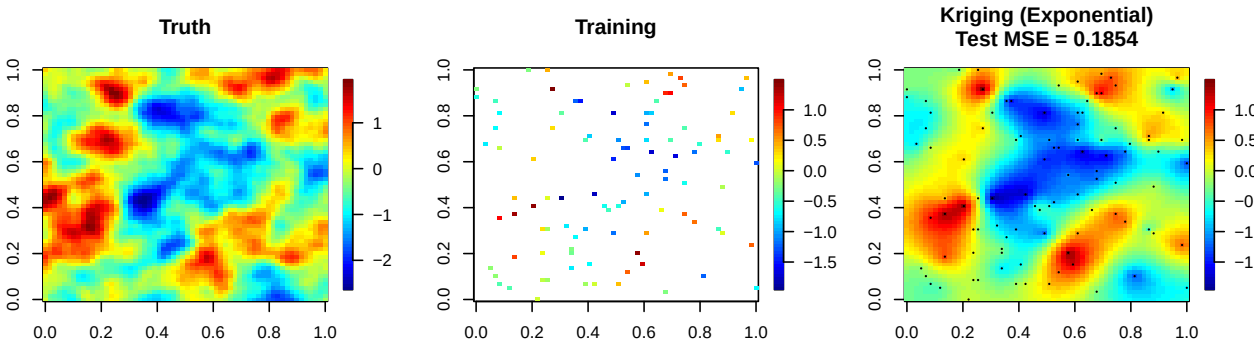
```r
dgrid <- seq(0, max(distance_matrix), length.out = 300)

# TRUE curve (simulation parameters)
cov_true <- sigma2_true * exp(-dgrid / beta_true)
cov_true[dgrid == 0] <- NA
nugget_true <- sigma2_true + tausq_true

# MLE parameters
tau2_mle   <- est_par[1]^2
sigma2_mle <- est_par[2]^2
beta_mle   <- exp(est_par[3])
nu <- 0.5

# MLE curve
cov_mle <- sigma2_mle * exp(-dgrid/beta_mle)

cov_mle[dgrid == 0] <- NA
nugget_mle <- sigma2_mle + tau2_mle

# Y limits
ymax <- max(c(cov_true, cov_mle, nugget_true, nugget_mle), na.rm = TRUE)
ymin <- min(c(cov_true, cov_mle, 0), na.rm = TRUE)

plot(NA, xlim = range(dgrid), ylim = c(ymin, ymax),
     xlab = "Distance", ylab = "Covariance",
     main = "Exponential Covariance: True vs MLE")

lines(dgrid, cov_true, lwd = 2, col = "darkgreen")
lines(dgrid, cov_mle,  lwd = 2, col = "red")

# Nugget points at h = 0
points(0, nugget_true, pch = 15, col = "darkgreen", cex = 1.2)
points(0, nugget_mle,  pch = 17, col = "red",       cex = 1.2)

legend("topright",
       legend = c("True Exponential", "MLE Exponential",
                  expression(sigma^2 + tau^2~"(True)"),
                  expression(sigma^2 + tau^2~"(MLE)")),
       lty    = c(1, 1, NA, NA),
       pch    = c(NA, NA, 15, 17),
       col    = c("darkgreen", "red", "darkgreen", "red"),
       bty    = "n")
```




```r
#------------------ KRIGING PHASE ----------------------------------

# Build Sigma(\hat{theta}) for training data (retrieve MLE estiamtes again)
tau2_hat   <- est_par[1]^2
sigma2_hat <- est_par[2]^2
beta_hat   <- exp(est_par[3])
mu_hat     <- est_par[4]

# Covariance matrix using exponential model
exp_cov <- function(D, sigma2, beta) {
  sigma2 * exp(-D / beta)
}

C_tt <- exp_cov(train_D, sigma2_hat, beta_hat)
diag(C_tt) <- sigma2_hat + tau2_hat + 1e-10   # ensure numerical stability

# Find Explicit inverse for kriging
Sigma_inv <- solve(C_tt)

# Get Design matrix for mean m(s) (intercept-only model)
M <- matrix(1, nrow = n.train, ncol = 1)
beta_hat_gls <- solve(t(M) %*% Sigma_inv %*% M) %*%
  (t(M) %*% Sigma_inv %*% matrix(train_Z, ncol = 1))
```

```r
# Cross-covariances K(\hat{theta}) and predictions at all grid location
K_st <- exp_cov(D_all_train, sigma2_hat, beta_hat)
K_st[D_all_train == 0] <- sigma2_hat

m0 <- matrix(1, nrow = n.loc, ncol = 1)
resid <- matrix(train_Z, ncol = 1) - M %*% beta_hat_gls
pred_all <- as.vector(m0 %*% beta_hat_gls + K_st %*% (Sigma_inv %*% resid))

# Kriging variances
C_ss_diag <- rep(sigma2_hat + tau2_hat, n.loc)
tmp <- K_st %*% Sigma_inv
var_all <- C_ss_diag - rowSums(tmp * K_st)

# Test MSE on held-out points
mse_test <- mean((pred_all[test_idx] - Z[test_idx])^2)
```

```r
# Plots (Truth | Training | Prediction + MSE)
par(mfrow = c(1,3))
quilt.plot(locs[,1], locs[,2], Z, nx = length(x),
           ny = length(y), main = "Truth")
quilt.plot(train_locs[,1], train_locs[,2], train_Z, nx = length(x),
           ny = length(y), main = "Training")
quilt.plot(locs[,1], locs[,2], pred_all, nx = length(x), ny = length(y),
           main = paste0("Kriging (Exponential)\nTest MSE = ",
                         signif(mse_test, 4)))
points(train_locs[,1], train_locs[,2], pch = 16, cex = 0.3)
```




```r
print(paste("Exponential Covariance MSE:", round(mse_test,4)))
```

```
## [1] "Exponential Covariance MSE: 0.1854"
```

---

3 Gaussian

```
# MLE Estimation, Gaussian
#------------------- GAUSSIAN ---------------------------------------
# Define negative log-likelihood function for MLE estimation via "optim"
nll_gaussian <- function(par) {
  tausq   <- par[1]^2          # nugget variance (forced gte 0)
  sigmasq <- par[2]^2          # partial sill
  beta    <- exp(par[3])       # range parameter (gt 0)
  mu_val  <- par[3]            # constant mean

  # monitor parameter progress
  cat(sprintf("tau^2 = %.4f, sigma^2 = %.4f,
              beta = %.4f, mu = %.4f\n",
              tausq, sigmasq, beta, nu, mu_val))

  mu <- rep(mu_val, n.loc)

  dist_mat <- train_D
  # Build covariance matrix
  C <- sigmasq * exp(-(dist_mat / beta)^2)
  diag(C) <- sigmasq + tausq

  # Cholesky decomposition for log-likelihood
  L <- t(chol(C))
  log.det.cov <- 2 * sum(log(diag(L)))
  z.new <- forwardsolve(L, train_Z - mu)
  inner.prod <- sum(z.new^2)

  # Negative Gaussian log-likelihood
  nll_value <- 0.5 * n.loc * log(2*pi) + 0.5 * log.det.cov + 0.5 * inner.prod
  return(nll_value)
}
#------------------------------------------------------------------#
# Fit parameters by minimizing NLL
init <- c(0.01, 1, log(0.1), 0)
```

```
fit <- optim(init, nll_gaussian, control = list(trace = 0, maxit = 5000))
```

```
# Verify Convergence
str(fit)
```

```
## List of 5
##  $ par        : num [1:4] 0.207 1.503 -1.866 1.838
##  $ value      : num 3319
```

```
##  $ counts     : Named int [1:2] 173 NA
##   ..- attr(*, "names")= chr [1:2] "function" "gradient"
##  $ convergence: int 0
##  $ message    : NULL
```

```
# Extract and print true vs estimated
est_par <- fit$par

results <- data.frame(
  Parameter = c("tau^2", "sigma^2", "beta (range)", "mu (mean)"),
  True = c(tausq_true, sigma2_true, beta_true, 0),
  Estimated = c(
    est_par[1]^2,
    est_par[2]^2,
    exp(est_par[3]),
    est_par[4]
  )
)

print("True vs. Estimated Parameters:")
```

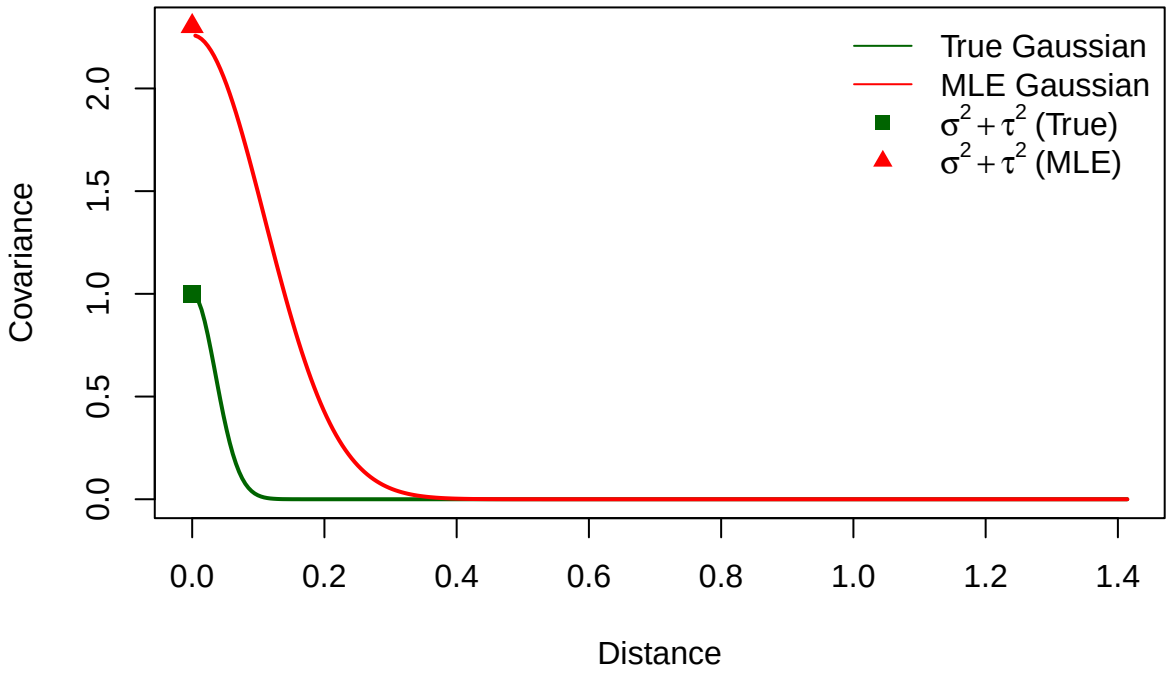
```
## [1] "True vs. Estimated Parameters:"
```

```
print(results, digits = 4, row.names = FALSE)
```

```
##     Parameter True Estimated
##         tau^2 0.00   0.04296
##       sigma^2 1.00   2.25942
##  beta (range) 0.05   0.15474
##     mu (mean) 0.00   1.83843
```

```
# Plot Gaussian covariance - True vs MLE

# Distance grid for curves
dgrid <- seq(0, max(distance_matrix), length.out = 300)

# TRUE curve (simulation parameters)
cov_true <- sigma2_true * exp(-(dgrid / beta_true)**2)
cov_true[dgrid == 0] <- NA
nugget_true <- sigma2_true + tausq_true

# MLE parameters
tau2_mle   <- est_par[1]^2
sigma2_mle <- est_par[2]^2
beta_mle   <- exp(est_par[3])

# MLE curve
```

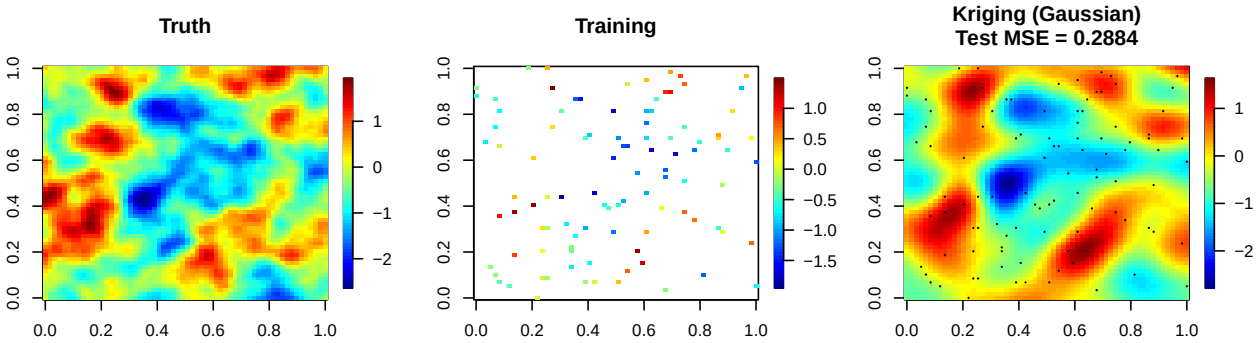
```r
cov_mle <- sigma2_mle * exp(-(dgrid/beta_mle)**2)
cov_mle[dgrid == 0] <- NA
nugget_mle <- sigma2_mle + tau2_mle

# Y limits
ymax <- max(c(cov_true, cov_mle, nugget_true, nugget_mle), na.rm = TRUE)
ymin <- min(c(cov_true, cov_mle, 0), na.rm = TRUE)

plot(NA, xlim = range(dgrid), ylim = c(ymin, ymax),
     xlab = "Distance", ylab = "Covariance",
     main = "Gaussian Covariance: True vs MLE")

lines(dgrid, cov_true, lwd = 2, col = "darkgreen")
lines(dgrid, cov_mle,  lwd = 2, col = "red")

# Nugget points at h = 0
points(0, nugget_true, pch = 15, col = "darkgreen", cex = 1.2)
points(0, nugget_mle,  pch = 17, col = "red",       cex = 1.2)

legend("topright",
       legend = c("True Gaussian", "MLE Gaussian",
                  expression(sigma^2 + tau^2~"(True)"),
                  expression(sigma^2 + tau^2~"(MLE)")),
       lty    = c(1, 1, NA, NA),
       pch    = c(NA, NA, 15, 17),
       col    = c("darkgreen", "red", "darkgreen", "red"),
       bty    = "n")
```




```r
#------------------ KRIGING PHASE ----------------------------------

# Extract MLE parameter estimates
tau2_hat   <- est_par[1]^2
sigma2_hat <- est_par[2]^2
beta_hat   <- exp(est_par[3])
mu_hat     <- est_par[4]

# Define Gaussian covariance function
gaussian_cov <- function(D, sigma2, beta) {
  sigma2 * exp(-(D / beta)^2)
}

# Build covariance matrix for training data
C_tt <- gaussian_cov(train_D, sigma2_hat, beta_hat)
diag(C_tt) <- sigma2_hat + tau2_hat + 1e-10   # ensure positive definiteness

# Compute inverse for kriging
Sigma_inv <- solve(C_tt)

# Design matrix for mean m(s) (intercept-only model)
M <- matrix(1, nrow = n.train, ncol = 1)
beta_hat_gls <- solve(t(M) %*% Sigma_inv %*% M) %*%
  (t(M) %*% Sigma_inv %*% matrix(train_Z, ncol = 1))
```

```r
# Cross-covariances k(\hat{theta}) and predictions at all grid location
K_st <- gaussian_cov(D_all_train, sigma2_hat, beta_hat)
K_st[D_all_train == 0] <- sigma2_hat

m0 <- matrix(1, nrow = n.loc, ncol = 1)
resid <- matrix(train_Z, ncol = 1) - M %*% beta_hat_gls
pred_all <- as.vector(m0 %*% beta_hat_gls + K_st %*% (Sigma_inv %*% resid))

# Kriging variances
C_ss_diag <- rep(sigma2_hat + tau2_hat, n.loc)
tmp <- K_st %*% Sigma_inv
var_all <- C_ss_diag - rowSums(tmp * K_st)

# Test MSE on held-out points
mse_test <- mean((pred_all[test_idx] - Z[test_idx])^2)
```

```r
# Visualization
par(mfrow = c(1,3))
quilt.plot(locs[,1], locs[,2], Z, nx = length(x), ny = length(y),
           main = "Truth")
quilt.plot(train_locs[,1], train_locs[,2], train_Z, nx = length(x),
           ny = length(y), main = "Training")
quilt.plot(locs[,1], locs[,2], pred_all, nx = length(x), ny = length(y),
           main = paste0("Kriging (Gaussian)\nTest MSE = ",
                         signif(mse_test, 4)))
points(train_locs[,1], train_locs[,2], pch = 16, cex = 0.3)
```




```r
print(paste("Gaussian Covariance MSE:", round(mse_test,4)))
```

```
## [1] "Gaussian Covariance MSE: 0.2884"
```

---

4 Matern

```r
# MLE Estimation, Matern
#------------------- MATERN -----------------------------------------
# Define negative log-likelihood function for MLE estimation via "optim"
nll_matern <- function(par) {
  tausq   <- par[1]^2          # nugget variance (forced gte 0)
  sigmasq <- par[2]^2          # partial sill
  beta    <- exp(par[3])       # range parameter (gt 0)
  nu      <- par[4]            # smoothness
  mu_val  <- par[5]            # constant mean

  # monitor parameter progress
  cat(sprintf("tau^2 = %.4f, sigma^2 = %.4f,
              beta = %.4f, nu = %.4f, mu = %.4f\n",
              tausq, sigmasq, beta, nu, mu_val))

  mu <- rep(mu_val, n.loc)

  dist_mat <- train_D
  # Build covariance matrix
  C <- (sigmasq / (2^(nu - 1) * gamma(nu))) *
      (dist_mat / beta)^nu * besselK(dist_mat / beta, nu)
  diag(C) <- sigmasq + tausq

  # Cholesky decomposition for log-likelihood
  L <- t(chol(C))
  log.det.cov <- 2 * sum(log(diag(L)))
  z.new <- forwardsolve(L, train_Z - mu)
  inner.prod <- sum(z.new^2)

  # Negative Gaussian log-likelihood
  nll_value <- 0.5 * n.loc * log(2*pi) + 0.5 * log.det.cov + 0.5 * inner.prod
  return(nll_value)
}
#------------------------------------------------------------------#

# Fit parameters by minimizing NLL
init <- c(0.1, 1, log(0.1), 0.8, 0)
```

```r
fit <- optim(init, nll_matern, control = list(trace = 0, maxit = 5000))
```

```r
# Verify Convergence
str(fit)
```

```
## List of 5
##  $ par        : num [1:5] -0.000121 0.807219 -3.203856 2.267827 -0.251631
##  $ value      : num 3291
##  $ counts     : Named int [1:2] 242 NA
##   ..- attr(*, "names")= chr [1:2] "function" "gradient"
##  $ convergence: int 0
##  $ message    : NULL
```

```
# Extract and print true vs estimated
est_par <- fit$par

results <- data.frame(
  Parameter = c("tau^2", "sigma^2", "beta (range)",
                "nu (smoothness)", "mu (mean)"),
  True = c(tausq_true, sigma2_true, beta_true, nu_true, 0),
  Estimated = c(
    est_par[1]^2,
    est_par[2]^2,
    exp(est_par[3]),
    est_par[4],
    est_par[5]
  )
)

print("True vs. Estimated Parameters:")
```

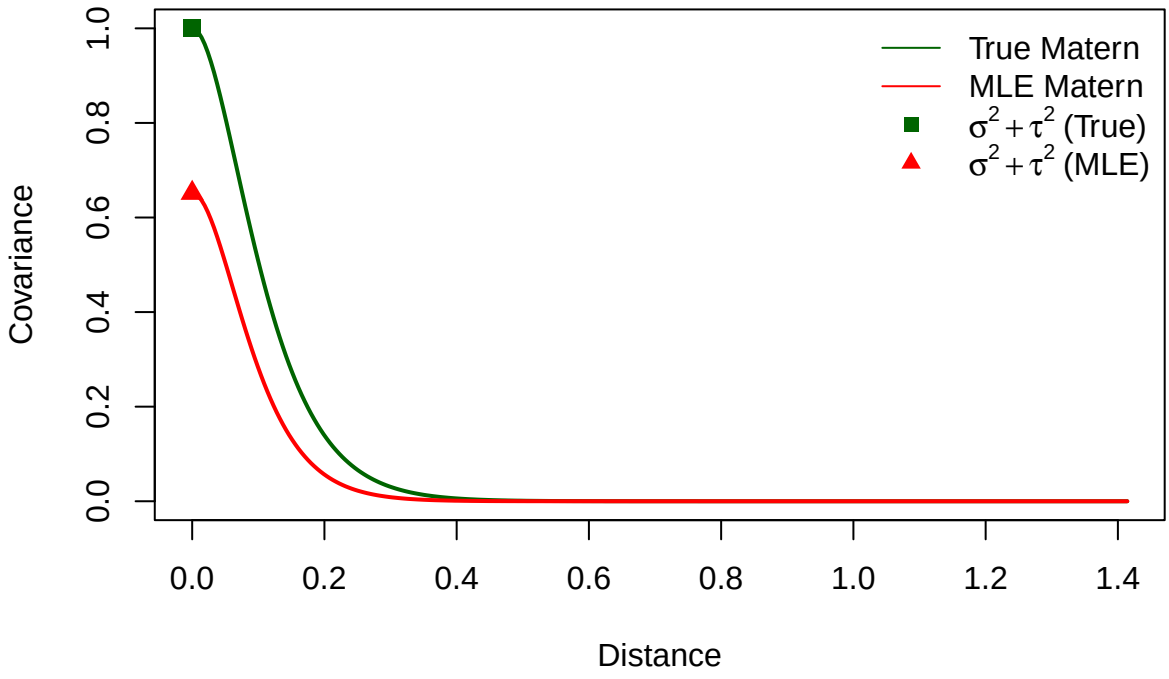
```
## [1] "True vs. Estimated Parameters:"
```

```
print(results, digits = 4, row.names = FALSE)
```

```
##        Parameter True  Estimated
##            tau^2 0.00  1.469e-08
##          sigma^2 1.00  6.516e-01
##     beta (range) 0.05  4.061e-02
##  nu (smoothness) 2.00  2.268e+00
##        mu (mean) 0.00 -2.516e-01
```

```
# Plot Matern covariance - True vs MLE

# Distance grid for curves
dgrid <- seq(0, max(distance_matrix), length.out = 300)

# TRUE curve (simulation parameters)
cov_true <- (sigma2_true / (2^(nu_true - 1) * gamma(nu_true))) *
  (dgrid / beta_true)^nu_true * besselK(dgrid / beta_true, nu_true)
cov_true[dgrid == 0] <- NA
nugget_true <- sigma2_true + tausq_true
```

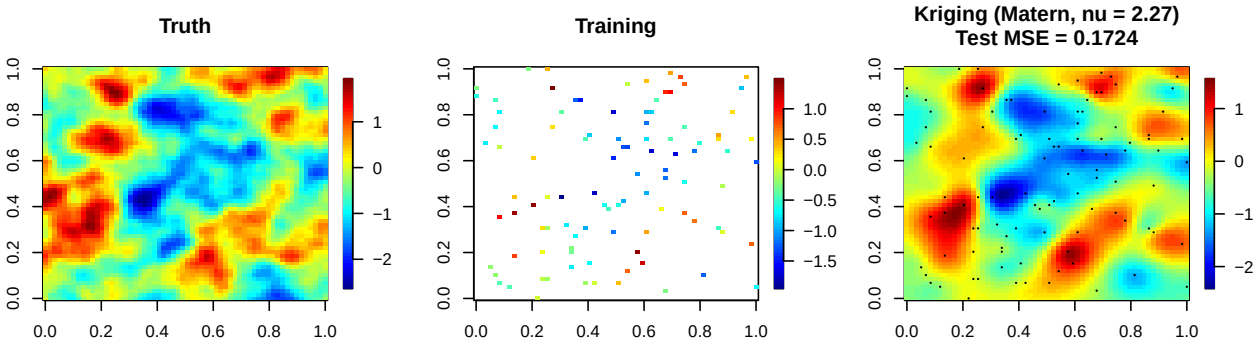
```r
# MLE parameters
tau2_mle   <- est_par[1]^2
sigma2_mle <- est_par[2]^2
beta_mle   <- exp(est_par[3])
nu_mle     <- est_par[4]

# MLE curve
cov_mle <- (sigma2_mle / (2^(nu_mle - 1) * gamma(nu_mle))) *
  (dgrid / beta_mle)^nu_mle * besselK(dgrid / beta_mle, nu_mle)
cov_mle[dgrid == 0] <- NA
nugget_mle <- sigma2_mle + tau2_mle

# Y limits
ymax <- max(c(cov_true, cov_mle, nugget_true, nugget_mle), na.rm = TRUE)
ymin <- min(c(cov_true, cov_mle, 0), na.rm = TRUE)

plot(NA, xlim = range(dgrid), ylim = c(ymin, ymax),
     xlab = "Distance", ylab = "Covariance",
     main = "Matern Covariance: True vs MLE")

lines(dgrid, cov_true, lwd = 2, col = "darkgreen")
lines(dgrid, cov_mle,  lwd = 2, col = "red")

# Nugget points at h = 0
points(0, nugget_true, pch = 15, col = "darkgreen", cex = 1.2)
points(0, nugget_mle,  pch = 17, col = "red",       cex = 1.2)

legend("topright",
       legend = c("True Matern", "MLE Matern",
                  expression(sigma^2 + tau^2~"(True)"),
                  expression(sigma^2 + tau^2~"(MLE)")),
       lty    = c(1, 1, NA, NA),
       pch    = c(NA, NA, 15, 17),
       col    = c("darkgreen", "red", "darkgreen", "red"),
       bty    = "n")
```




```r
#------------------ KRIGING PHASE ----------------------------------

# Extract MLE parameter estimates
tau2_hat   <- est_par[1]^2
sigma2_hat <- est_par[2]^2
beta_hat   <- exp(est_par[3])
nu_hat     <- est_par[4]
mu_hat     <- est_par[5]

# Define Matern covariance function
matern_cov <- function(D, sigma2, beta, nu) {
  # Handle zero distances to avoid NaN from BesselK
  D[D == 0] <- 1e-10
  cov_val <- (sigma2 / (2^(nu - 1) * gamma(nu))) *
    (D / beta)^nu * besselK(D / beta, nu)
  return(cov_val)
}

# Build covariance matrix for training data
C_tt <- matern_cov(train_D, sigma2_hat, beta_hat, nu_hat)
diag(C_tt) <- sigma2_hat + tau2_hat + 1e-10   # ensure positive definiteness

# Compute inverse for kriging
Sigma_inv <- solve(C_tt)
```

```r
# Design matrix for mean m(s) (intercept-only model)
M <- matrix(1, nrow = n.train, ncol = 1)
beta_hat_gls <- solve(t(M) %*% Sigma_inv %*% M) %*%
  (t(M) %*% Sigma_inv %*% matrix(train_Z, ncol = 1))

# Cross-covariances k(\hat{theta}) and predictions at all grid location
K_st <- matern_cov(D_all_train, sigma2_hat, beta_hat, nu_hat)
K_st[D_all_train == 0] <- sigma2_hat

m0 <- matrix(1, nrow = n.loc, ncol = 1)
resid <- matrix(train_Z, ncol = 1) - M %*% beta_hat_gls
pred_all <- as.vector(m0 %*% beta_hat_gls + K_st %*% (Sigma_inv %*% resid))

# Kriging variances
C_ss_diag <- rep(sigma2_hat + tau2_hat, n.loc)
tmp <- K_st %*% Sigma_inv
var_all <- C_ss_diag - rowSums(tmp * K_st)

# Test MSE on held-out points
mse_test <- mean((pred_all[test_idx] - Z[test_idx])^2)
```

```r
# Visualization
par(mfrow = c(1,3))
quilt.plot(locs[,1], locs[,2], Z, nx = length(x), ny = length(y),
           main = "Truth")
quilt.plot(train_locs[,1], train_locs[,2], train_Z, nx = length(x),
           ny = length(y), main = "Training")
quilt.plot(locs[,1], locs[,2], pred_all, nx = length(x), ny = length(y),
           main = paste0("Kriging (Matern, nu = ", round(nu_hat, 2),
                         ")\nTest MSE = ", signif(mse_test, 4)))
points(train_locs[,1], train_locs[,2], pch = 16, cex = 0.3)
```




```r
print(paste("Matern Covariance MSE:", round(mse_test, 4)))
```

```
## [1] "Matern Covariance MSE: 0.1724"
```

5 Concluding Question: What do you observe about the kriging predictions under the Exponential, Gaussian, and Matern models? Which performs best based on the MSE, and why?

I ob

Special credit to Dr. Salvana for providing much of the relevant code for this assignment.